

## Tutorial 4: Map, filter, comprehension

We will practice the **higher-order** functions `map`, `filter`, `zip`, and `zipWith`, and simultaneously, **list comprehensions** (detailed on the next page). The function `map` takes as its first argument a function `f :: a -> b`, as its second argument a list `xs :: [a]`, and applies `f` to each element in `xs` to return a list of type `[b]`.

```
map :: (a -> b) -> [a] -> [b]
map _ []          = []
map f (x:xs)      = f x : map f xs
```

The function `filter` takes a **predicate** `p :: a -> Bool` (something that is true or false for a value of type `a`) and selects those elements from a list for which the predicate is true.

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ []          = []
filter p (x:xs)
  | p x              = x : filter p xs
  | otherwise        = filter p xs
```

The function `zip` combines two lists into a list of pairs. If one list is longer than the other, the remaining elements are discarded.

```
zip :: [a] -> [b] -> [(a,b)]
zip _ [] = []
zip [] _ = []
zip (x:xs) (y:ys) = (x,y) : zip xs ys
```

The function `zipWith` is similar, except that instead of forming pairs, it combines the two lists with an arbitrary function.

```
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith f _ [] = []
zipWith f [] _ = []
zipWith f (x:xs) (y:ys) = f x y : zipWith f xs ys
```

A **list comprehension** is a convenient notation for list operations:

```
ghci> [ 3*n + 1 | n <- [0..10] , even n ]
[1,7,13,19,25,31]
```

The meaning of this example is as follows:

- Construct the list of all numbers  $3*n + 1$ , where
- $n$  is an element of the list  $[1..10]$ , and
- $n$  is even.

The syntax is designed after the way sets are described in mathematics. The following describes the set of all numbers  $3n+1$  where  $n$  is a natural number and  $n$  is even:

$$\{3n + 1 \mid n \in \mathbb{N}, n \text{ is even}\}$$

A list comprehension is constructed as follows:

```
[ exp | qualifier_1 , ... , qualifier_n ]
```

where each qualifier can be a **generator**  $x \leftarrow xs$  that **draws** elements  $x$  from a list  $xs$  (such as  $n \leftarrow [1..]$  above), or a boolean **guard** (such as  $\text{prime } n$  above), which filters out cases that do not pass the given test. List comprehensions are intimately related to maps and filters:

```
map f (filter p xs) == [ f x | x <- xs , p x ]
```

**Exercise 1:** Write three versions of each of the following functions, one using **recursion**, one using **map** and **filter**, and one using **list comprehension**.

- A function `doubles` that multiplies every number in a list by two. It may be helpful to create a function `double` to double a single number.
- A function `multiplesOfThree` that removes any numbers from a list that are not a multiple of three. Use the built-in `mod`, and define an auxiliary function to test divisibility by three.
- A function `doubleMultiplesOfThree` that takes the multiples of three from a list (removing the others) and then doubles them.

```
ghci> doubles [1..10]
[2,4,6,8,10,12,14,16,18,20]
ghci> multiplesOfThree [1..10]
[3,6,9]
ghci> doubleMultiplesOfThree [1..10]
[6,12,18]
```

## Where-clauses

When you use auxiliary functions such as `double` that are only used by one function, and not needed elsewhere in your program, it is good practice to put them in a **where-clause**. For instance, for a function that squares each number in a list:

```
squareAll :: [Int] -> [Int]
squareAll xs = map square xs
  where
    square x = x*x
```

The function `square` is now **local** to the definition of `squareAll`. Other functions cannot use it, nor can you use it when you load this into `ghci`. Firstly, this tidies up your code for better readability. Secondly, if your function uses the same value multiple times, if you put it in a where-clause then you are certain it will be computed only once.

A where-clause is part of the syntax of function definitions. Each pattern-matching case can have its own where-clause. Guards do not have separate where-clauses, but a single one for all guards in a given pattern-matching case.

## Exercise 2:

- Tidy up the functions of the previous exercise by putting any auxiliary functions in where-clauses.
- Complete the function `shorts` that removes all strings of 4 characters or more from a list. Use `filter`, and a where-clause to define the filtering property. Write a second version using **list comprehension**.
- Complete the function `incrementPositives` that takes all positive integers in a list and adds one. Use `map` and `filter`, with a where-clause to define any auxiliary functions you might need. Write a second version using **list comprehension**.
- Complete the function `difference` which takes two strings, and removes every character occurring in the second, from the first. Use `filter`, and `elem` and `not` may be helpful. Write a second version using **list comprehension**.

- e) Complete the function `oddLengthSums` that given a list of integer lists, returns for each odd-length list its sum. Use `map` and `filter`, and `odd`, `length`, and `sum` may be helpful. Write a second version using **list comprehension**.
- f) Give the functions above their most general type using type variables and type classes.

```
ghci> shorts ["one","two","three","four","five","six","seven"]
["one","two","six"]
ghci> incrementPositives [-3,4,1,-2,0,3]
[5,2,4]
ghci> difference "difference" "ef"
"dirnc"
ghci> oddLengthSums [[1],[1,2],[1,2,3],[1..4],[1..5]]
[1,6,15]
```

## Zip and zipWith

### Exercise 3:

- a) Complete the function `numbered` that indexes the elements in a list by pairing them with numbers, counting up from 1. Use the function `zip`.
- b) Complete the function `everyother` that takes every other element from a list, starting with the first. **Hint:** use `numbered` to count elements, `filter`, `fst`, and `odd` to select those pairs to keep, and `map` and `snd` to remove the indexing again. Write a second version using **list comprehension**.
- c) Complete the function `same` that takes two lists and returns a list of the positions where their elements coincide. For instance, the strings `"Mary"` and `"Jane"` have the same 2nd characters, so `same` should return the list `[2]`. Use the functions `filter`, `map`, `fst`, `snd`, `zip` (or `numbered`), and `zipWith`. Write a second version using **list comprehension**, using `zip` and `zipWith`, or instead using `zip3 :: [a] -> [b] -> [c] -> [(a,b,c)]`.

```
ghci> numbered "days"
[(1,'d'),(2,'a'),(3,'y'),(4,'s')]
ghci> everyother "Elizabeth"
"Eiaeh"
ghci> same "Charles" "Charlotte"
[1,2,3,4,5]
```

## Extra

### Partial application, sections, function composition

Haskell has several further techniques to help readability (or to completely obfuscate your code, depending on how you use them).

**Partial application** is when you provide a function with some of its arguments, but not all. For instance, `max :: Int -> Int -> Int` is a function taking two integers; `max 3 5` returns the integer `5`; and `max 3 :: Int -> Int` is a function that takes one more integer, and then returns an integer.

To put this another way: function application takes one argument at a time, and is left-associative:

```
max 3 5 == (max 3) 5 == 5
```

**Sections** are partially-applied infix operators, written by putting parentheses around the operator and any given argument. For instance, `(+3)` is the function that adds three to any number; `(5*)` multiplies by five; `(<10)` checks if a number is less than ten; `(/=0)` if it is non-zero. With zero arguments, a section makes a prefix function: e.g. `(+)` is addition.

```
1 < 2 == (<) 1 2 == (1<) 2 == (<2) 1 == True
```

One can use sections with normal functions by making them infix with backticks, `'...'`:

```
mod 5 3 == (`mod` 3) 5 == (5 `mod`) 3 == 2
```

**Function composition**, written `f . g` to match the mathematical notation  $f \circ g$ , applies first `g` and then `f` to an argument. For instance, `(+1).length` is the function that takes the length of a list and adds one to that. Composition is defined as follows:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
(f . g) x = f (g x)
```

### Exercise 4:

- a) Use partial application, sections, and function composition to simplify your answers to the previous exercises.

## Multiple generators

List comprehensions with multiple generators are very useful in exploring possible combinations of list elements.

### Exercise 5:

- Complete the function `combinations` which takes two lists and produces every possible combination of items. Use a list comprehension with two generators.
- Complete the function `selfcombinations` which takes a list and produces every possible pairing of its items, avoiding symmetric duplicates. That is, for each element, pair it only with those coming **after** it in the list, and with itself. **Hint:** in your list comprehension, use `zip` to count elements and `drop` to obtain those after that index.
- Complete the function `pyts` so that `pyts n` generates all ordered Pythagorean triples with numbers up to (and including) `n`.

```
ghci> combinations "xy" [1,2,3]
[('x',1),('x',2),('x',3),('y',1),('y',2),('y',3)]
```

```
ghci> selfcombinations [1..4]
[(1,1),(1,2),(1,3),(1,4),(2,2),(2,3),(2,4),(3,3),(3,4),(4,4)]
```

```
ghci> pyts 100
[(3,4,5),(5,12,13),(6,8,10),(7,24,25),(8,15,17),(9,12,15),
(9,40,41),(10,24,26),(11,60,61),(12,16,20),(12,35,37),(13,84,85),
(14,48,50),(15,20,25),(15,36,39),(16,30,34),(16,63,65),(18,24,30),
(18,80,82),(20,21,29),(20,48,52),(21,28,35),(21,72,75),(24,32,40),
(24,45,51),(24,70,74),(25,60,65),(27,36,45),(28,45,53),(28,96,100),
(30,40,50),(30,72,78),(32,60,68),(33,44,55),(33,56,65),(35,84,91),
(36,48,60),(36,77,85),(39,52,65),(39,80,89),(40,42,58),(40,75,85),
(42,56,70),(45,60,75),(48,55,73),(48,64,80),(51,68,85),(54,72,90),
(57,76,95),(60,63,87),(60,80,100),(65,72,97)]
```

## Challenge

**Exercise 6:** Use list comprehensions to write the following combinatoric functions.

- The function `choice` which, given a list of lists, gives every possible way of selecting exactly one element from each sublist.
- The function `splits` which gives every way of splitting a list into a first and a second part, so that concatenating both parts returns the original list.
- The function `permutations` which gives every permutation (reordering) of the input list. Use your function `splits`.

```
ghci> choice ["abc","12345","XY"]
["a1X","a1Y","a2X","a2Y","a3X","a3Y","a4X","a4Y","a5X","a5Y",
 "b1X","b1Y","b2X","b2Y","b3X","b3Y","b4X","b4Y","b5X","b5Y",
 "c1X","c1Y","c2X","c2Y","c3X","c3Y","c4X","c4Y","c5X","c5Y"]
```

```
ghci> splits [1..5]
([],[1,2,3,4,5]),([1],[2,3,4,5]),([1,2],[3,4,5]),
 ([1,2,3],[4,5]),([1,2,3,4],[5]),([1,2,3,4,5],[ ])]
```

```
ghci> permutations "abcde"
["abcde","bacde","bcade","bcdae","bcdea","acbde","cabde",
 "cbade","cbdae","cbdea","acdbe","cadbe","cdabe","cdbae",
 "cdbea","acdeb","cadeb","cdaeb","cdeab","cdeba","abdce",
 "badce","bdace","bdcae","bdcea","adbce","dabce","dbace",
 "dbcae","dbcea","adcbe","dacbe","dcabe","dcbae","dcbea",
 "adceb","daceb","dcaeb","dceab","dceba","abdec","badec",
 "bdaec","bdeac","bdeca","adbec","dabec","dbaec","dbeac",
 "dbeca","adebc","daebc","deabc","debac","debca","adecb",
 "daecb","deacb","decab","decba","abced","baced","bcaed",
 "bcead","bceda","acbed","cabed","cbaed","cbead","cbeda",
 "acebd","caebd","ceabd","cebad","cebda","acedb","caedb",
 "ceadb","cedab","cedba","abecd","baecd","beacd","becad",
 "becda","aebcd","eabcd","ebacd","ebcad","ebcda","aecbd",
 "eacbd","ecabd","ecbad","ecbda","aecdb","eacdb","ecadb",
 "ecdab","ecdba","abedc","baedc","beadc","bedac","bedca",
 "aebdc","eabdc","ebadc","ebdac","ebdca","aedbc","eadbc",
 "edabc","edbac","edbca","aedcb","eadcb","edacb","edcab",
 "edcba"]
```